Lecture # 14

By: Muhammad Zain Uddin

email: zuddin@iba.edu.pk

Revision

- What are sockets?
- Types of sockets
- How do sockets work?
- Socketpair

Attribution

TODAY'S LECTURE IS BASED ON ADVANCED PROGRAMMING IN THE UNIX® ENVIRONMENT

Agenda

Interprocess Communication

- Message Queues

What are Message Queues?

A **Message Queue** is a linked list of messages stored within the kernel and identified by a message queue identifier

A **message queue** is kind of like a FIFO (First In First Out) pipe... but with some advantages.

- It allows processes to **send and receive structured messages**.
- Unlike FIFOs (which are just byte streams), message queues:
  - Retain **message boundaries**
  - Allow **priority-based retrieval** (i.e., you can pull a specific message out of the queue even if it's not at the front — think of it as legally cutting in line).

How Message Queues Work?

A new queue is created or an existing queue opened by **msgget**.

New messages are added to the end of a queue by **msgsnd**.

Every message has

- a positive long integer type field,
- a non-negative length, and
- the actual data bytes (corresponding to the length), all of which are specified to msgsnd when the message is added to a queue.

Messages are fetched from a queue by **msgrcv**.

We don’t have to fetch the messages in a first-in, first-out order.

- Instead, we can fetch messages based on their type field.

Message Queue Structure

Each queue has the following **msqid_ds** structure associated with it:

```
struct msqid_ds {
    struct ipc_perm msg_perm; /* permission structure*/
    msgqnum_t msg_qnum; /* # of messages on queue */
    msglen_t msg_qbytes; /* max # of bytes on queue */
    pid_t msg_lspid; /* pid of last msgsnd() */
    pid_t msg_lrpid; /* pid of last msgrcv() */
    time_t msg_stime; /* last-msgsnd() time */
    time_t msg_rtime; /* last-msgrcv() time */
    time_t msg_ctime; /* last-change time */

};
```

Creating a new queue

The first function normally called is msgget to either open an existing queue or create a new queue.

```
#include <sys/msg.h>

int msgget(key_t key, int flag);
```

Returns: message queue ID if OK, –1 on error

When a new queue is created, the following members of the msqid_ds structure are initialized.

- The ipc_perm structure is initialized.
- msg_qnum, msg_lspid, msg_lrpid, msg_stime, and msg_rtime are all set to 0.
- msg_ctime is set to the current time.
- msg_qbytes is set to the system limit.

On success, msgget returns the non-negative queue ID.

- This value is then used with the other three message queue functions.

Key and Identifier

Each IPC structure (message queue, semaphore, or shared memory segment) in the kernel is referred to by a non-negative integer identifier.

Unlike file descriptors, IPC identifiers are not small integers.

The identifier is an internal name for an IPC object. Cooperating processes need an external naming scheme to be able to rendezvous using the same IPC object.

- For this purpose, an IPC object is associated with a key that acts as an external name

Whenever an IPC structure is being created, a key must be specified.

- The data type of this key is the primitive system data type **key_t**, which is often defined as a long integer.
- This key is converted into an identifier by the kernel.

Key and Identifier

There are various ways for a client and a server to rendezvous at the same IPC structure.

The server can create a new IPC structure by specifying a key of IPC_PRIVATE and store the returned identifier somewhere (such as a file) for the client to obtain.

The client and the server can agree on a key by defining the key in a common header, for example. The server then creates a new IPC structure specifying this key.

The client and the server can agree on a pathname and project ID (the project ID is a character value between 0 and 255) and call the function ftok to convert these two values into a key.

- The path argument must refer to an existing file.

msgctl function

The msgctl function performs various operations on a queue

```
#include <sys/msg.h>

int msgctl(int msqid, int cmd, struct msqid_ds *buf);
```

Returns: 0 if OK, –1 on error

IPC_STAT	Fetch the msqid_ds structure for this queue, storing it in the structure pointed to by buf.
IPC_SET	Copy the following fields from the structure pointed to by buf to the msqid_ds structure associated with this queue: msg_perm.uid, msg_perm.gid, msg_perm.mode, and msg_qbytes. This command can be executed only by a process whose effective user ID equals msg_perm.cuid or msg_perm.uid or by a process with superuser privileges.
IPC_RMID	Remove the message queue from the system and any data still on the queue. This removal is immediate. Any other process still using the message queue will get an error of EIDRM on its next attempted operation on the queue.

msgsnd

Data is placed onto a message queue by calling msgsnd.

```
#include <sys/msg.h>

int msgsnd(int msqid, const void *ptr, size_t nbytes, int flag);
```

Returns: 0 if OK, –1 on error

Each message is composed of

- a positive long integer type field,
- a non-negative length (nbytes), and
- the actual data bytes (corresponding to the length).

Messages are always placed at the end of the queue.

msgsnd

The ptr argument points to a long integer that contains the positive integer message type, and it is immediately followed by the message data. (There is no message data if nbytes is 0.)

If the largest message we send is 512 bytes, we can define the following structure:

```
struct mymesg {
  long  mtype;      /* positive message type */
  char  mtext[512]; /* message data, of length nbytes */
};
```

The ptr argument is then a pointer to a mymesg structure.

The message type can be used by the receiver to fetch messages in an order other than first in, first out.

Question

WHY IS LENGTH OFTEN EXPLICITLY PROVIDED IN SYSTEM CALLS?

INT MSGSND(INT MSQID, CONST VOID *PTR, SIZE_T NBYTES, INT FLAG);

Question

DO YOU SEE ANY ISSUE CONSIDERING THAT MSGSND IS A BLOCKING CALL?

IPC_NOWAIT

A flag value of IPC_NOWAIT can be specified in msgsnd call.

If the message queue is full (either the total number of messages on the queue equals the system limit, or the total number of bytes on the queue equals the system limit), specifying IPC_NOWAIT causes msgsnd to return immediately with an error of EAGAIN.

If IPC_NOWAIT is not specified, we are blocked until there is room for the message, the queue is removed from the system, or a signal is caught and the signal handler returns.

- In the second case, an error of EIDRM is returned (‘‘identifier removed’’);
- In the last case, the error returned is EINTR.

msgrcv

When msgsnd returns successfully, the msqid_ds structure associated with the message queue is updated to indicate the process ID that made the call (msg_lspid), the time that the call was made (msg_stime), and that one more message is on the queue (msg_qnum).

Messages are retrieved from a queue by msgrcv.

```
#include <sys/msg.h>

ssize_t msgrcv(int msqid, void *ptr, size_t nbytes, long type, int flag);
```

Returns: size of data portion of message if OK, –1 on error

msgsnd

As with msgsnd, the ptr argument points to a long integer (where the message type of the returned message is stored) followed by a data buffer for the actual message data.

nbytes specifies the size of the data buffer.

- If the returned message is larger than nbytes and the MSG_NOERROR bit in flag is set, the message is truncated. (In this case, no notification is given to us that the message was truncated, and the remainder of the message is discarded.)
- If the message is too big and this flag value is not specified, an error of E2BIG is returned instead (and the message stays on the queue).

Message priority

The type argument lets us specify which message we want.

type == 0 The first message on the queue is returned.

type > 0 The first message on the queue whose message type equals type is returned.

type < 0 The first message on the queue whose message type is the lowest value less than or equal to the absolute value of type is returned.

Question

CAN YOU FIND AN ALTERNATE USE FOR THE TYPE FIELD?

Type field – alternate use case

Another use of the type field is to contain the process ID of the client if a single message queue is being used by multiple clients and a single server

Process ID should fit in the long integer of the type field

IPC_NOWAIT flag

We can specify a flag value of IPC_NOWAIT to make the operation nonblocking, causing msgrcv to return −1 with errno set to ENOMSG if a message of the specified type is not available.

If IPC_NOWAIT is not specified, the operation blocks until

- a message of the specified type is available,
- the queue is removed from the system (−1 is returned with errno set to EIDRM),
- a signal is caught and the signal handler returns (causing msgrcv to return −1 with errno set to EINTR)

msgrcv

When msgrcv succeeds, the kernel updates the msqid_ds structure associated with the message queue to indicate the caller's process ID (msg_lrpid), the time of the call (msg_rtime), and that one less message is on the queue (msg_qnum).

Question

HOW ARE QUEUE REMOVAL HANDLED IN CONTRAST TO FILES?

Queue Removal

Since a reference count is not maintained with each message queue (as there is for open files), the removal of a queue simply generates errors on the next queue operation by processes still using the queue.

- Semaphores handle this removal in the same fashion.

In contrast, when a file is removed, the file's contents are not deleted until the last open descriptor for the file is closed.

Performance comparison

timing comparison of three of these techniques on Solaris

- The tests consisted of a program that created the IPC channel, called fork, and then sent about 200 megabytes of data from the parent to the child. The data was sent using 100,000 calls to msgsnd, with a message length of 2,000 bytes for the message queue, and 100,000 calls to write, with a length of 2,000 bytes for the full-duplex pipe and UNIX domain socket. The times are all in seconds.

Operation	User	System	Clock
message queue	0.58	4.16	5.09
full-duplex pipe	0.61	4.30	5.24
UNIX domain socket	0.59	5.58	7.49

Question

WHICH IS THE OLDER TECHNOLOGY: QUEUES OR SOCKETS?

Limitation

Message Queue Persistence

Limited Message Queue Resources

- The system has limits:
  - Max number of queues
  - Max total memory used by queues
  - Max size of a single message

Performance Bottlenecks

- All messages go through the kernel, which involves:
- Copying from user to kernel space
- Managing message headers and permissions

Comparison

Feature	Pipes	FIFOs	Message Queues
Communication Type	Unnamed, typically between parent and child	Named, between unrelated processes	Named (via key), between unrelated processes
Persistence	Exists only during process lifetime	Exists in filesystem until removed	Exists until explicitly removed
Message Boundaries	No (stream-oriented)	No (stream-oriented)	Yes (message-oriented)
Blocking Behavior	Blocking by default	Blocking by default	Configurable (with IPC_NOWAIT)
Message Typing	Not supported	Not supported	Supported (msgtyp)